

USER QUESTION:

data parallel, formula 1,2,3,4不理解, 请用例子解释

ASSUMED CONTEXT FROM PREVIOUS DISCUSSION:

The user is referring to a section of a document titled "DATA PARALLELISM COMMUNICATION ANALYSIS" which contained the following formulas and notes:

--- START OF REFERENCED DOCUMENT ---

DATA PARALLELISM COMMUNICATION ANALYSIS

The total gradient size that needs to be communicated is:

- Gradients = Parameters $\approx$ num_layers * 16*h^2

During backward pass, these gradients are communicated in buckets (default 25MB). The communication time to all-reduce each bucket is:

$t_{comm} = t_{comm_bucket} = [\text{bucket_size} * 2 * (DP - 1)] / [DP * \text{peak_bw}]$

Note

For bandwidth calculations, we use the bus bandwidth formulas from the NCCL documentation. These formulas account for the specific communication patterns when calculating effective bandwidth between GPUs.

The computation time for backward pass is:

$t_{compute} = (4 * \text{num_tokens} * \text{num_params}) / \text{peak_flops}$

For effective overlap, we need:

$[t_{comm} / t_{compute}] = [(\text{num_params} * DP - 1) / (\text{num_tokens} * DP * \text{peak_bw})] \leq 1$

This ratio helps determine if communication will become a bottleneck in training. When the ratio is less than 1, communication can be fully overlapped with computation.

--- END OF REFERENCED DOCUMENT ---

RESPONSE: EXPLANATION OF DATA PARALLELISM FORMULAS 1, 2, 3, AND 4 WITH EXAMPLES

Let me explain these 4 formulas step-by-step using a concrete example to help you understand the communication analysis in data parallel training.

BACKGROUND SETUP FOR THE EXAMPLE

Let's assume we have the following configuration for training a large Transformer model:

- DP = 8 (We are using 8 GPUs for data parallelism)
- num_layers = 32 (The model has 32 Transformer layers)
- h = 4096 (The hidden dimension size of the model)
- num_tokens = 524,288 (The number of tokens processed per training step.
Example: batch size * sequence length = 32 * 16384)
- peak_bw = 900 GB/s (Peak GPU-to-GPU bandwidth, e.g., NVIDIA H100 NVLink)

- peak_flops = 1979 TFLOPs (Peak compute performance of an H100 GPU for BF16)

We will use these numbers in all calculations below.

FORMULA 1: TOTAL GRADIENT SIZE

The Formula:

$$\text{Gradients} = \text{Parameters} \approx \text{num_layers} * 16 * h^2$$

Explanation:

This formula estimates the total number of gradients that need to be synchronized across all GPUs in a data parallel training run. In data parallelism, each GPU has a full copy of the model. After the backward pass, each GPU computes gradients for all model parameters. These gradients must be averaged (reduced) across all GPUs to ensure every GPU has the same updated model. Therefore, the amount of gradient data to communicate is approximately equal to the number of model parameters. The formula " $\text{num_layers} * 16 * h^2$ " is a standard approximation for the parameter count in a Transformer model.

Example Calculation:

Let's calculate the total number of gradients (which equals the number of parameters).

$$h^2 = 4096 * 4096 = 16,777,216$$

Now plug into the formula:

$$\begin{aligned} \text{num_layers} * 16 * h^2 &= 32 * 16 * 16,777,216 \\ &= 32 * 268,435,456 \\ &= 8,589,934,592 \end{aligned}$$

Result:

The total number of gradients (and parameters) is approximately 8.59 billion (8.59e9). This is the total amount of gradient data that needs to be communicated across the 8 GPUs.

FORMULA 2: COMMUNICATION TIME FOR ONE BUCKET

The Formula:

$$t_{\text{comm_bucket}} = [\text{bucket_size} * 2 * (\text{DP} - 1)] / [\text{DP} * \text{peak_bw}]$$

Explanation:

During the backward pass, gradients are not sent all at once. To overlap communication with computation, they are grouped into buckets (e.g., 25MB buckets). The All-Reduce operation for one bucket happens in two phases: Reduce-Scatter and All-Gather. The total data transferred across the network for one bucket is " $2 * (\text{DP} - 1)$ " times the bucket size. This formula calculates the time it takes to complete the All-Reduce operation for a single bucket, given the number of GPUs (DP) and the peak bandwidth (peak_bw).

Example Calculation:

We use the standard bucket size of 25 MB. First, convert this to bytes:

$$\text{bucket_size} = 25 \text{ MB} = 25 * 1,000,000 = 25,000,000 \text{ bytes.}$$

Now plug in the numbers:

$$\text{DP} = 8$$

$$\text{peak_bw} = 900 \text{ GB/s} = 900 * 1,000,000,000 = 9e11 \text{ bytes/second}$$

First, calculate the numerator: $\text{bucket_size} * 2 * (\text{DP} - 1)$

$$\begin{aligned}
&= 25,000,000 * 2 * (8 - 1) \\
&= 25,000,000 * 2 * 7 \\
&= 25,000,000 * 14 \\
&= 350,000,000
\end{aligned}$$

Next, calculate the denominator: $DP * peak_bw$
 $= 8 * (9e11)$
 $= 7.2e12$

Finally, divide numerator by denominator:
 $t_comm_bucket = 350,000,000 / 7.2e12$
 $= 4.86e-5$ seconds

Result:

The communication time for one 25MB bucket is 4.86e-5 seconds, which is 48.6 microseconds. This is very fast.

FORMULA 3: BACKWARD PASS COMPUTATION TIME

The Formula:

$$t_compute = (4 * num_tokens * num_params) / peak_flops$$

Explanation:

This formula estimates the time spent on the backward pass computation for one training step. The backward pass is roughly twice as computationally intensive as the forward pass. The factor of 4 accounts for both the forward and backward passes in a standard transformer training iteration. "num_tokens" is the total number of tokens processed in the step, and "num_params" is the total number of model parameters. The product is divided by the peak floating-point operations per second (peak_flops) of the GPU to get the compute time.

Example Calculation:

First, we need num_params, which we calculated in Formula 1: $num_params = 8.59e9$.

We have $num_tokens = 524,288$.

$peak_flops = 1979$ TFLOPs $= 1979 * 1e12 = 1.979e15$ FLOPs/second.

Now, calculate the numerator: $4 * num_tokens * num_params$

$$= 4 * 524,288 * 8.59e9$$

$$\text{First, } 4 * 524,288 = 2,097,152$$

$$\text{Then, } 2,097,152 * 8.59e9 = 1.80e16 \text{ (This is the total FLOPs for the step).}$$

Now divide by peak_flops:

$$t_compute = 1.80e16 / 1.979e15$$

$$= 9.09 \text{ seconds}$$

Result:

The backward pass computation for one training step takes approximately 9.1 seconds on a single H100 GPU.

FORMULA 4: COMMUNICATION-TO-COMPUTATION RATIO FOR OVERLAP

The Formula (as derived and corrected for proper comparison):

$$\text{Ratio} = [t_comm_total / t_compute]$$

Where t_comm_total is the total communication time for all gradients.

A simplified and common form of the ratio is:

$$\left[\frac{(\text{num_params} * 2 * (\text{DP} - 1) / \text{DP}) / \text{peak_bw}}{[(4 * \text{num_tokens} * \text{num_params}) / \text{peak_flops}]} \right] \leq 1$$

This simplifies to:

$$[2 * (\text{DP} - 1) * \text{peak_flops}] / [4 * \text{DP} * \text{num_tokens} * \text{peak_bw}] \leq 1$$

Explanation:

This ratio is the key to understanding if communication can be effectively overlapped with computation. If the ratio is less than 1, the communication is faster than the computation. This means the GPU can finish communicating a bucket of gradients while it is still computing gradients for the next bucket, effectively hiding the communication latency. If the ratio is greater than 1, communication is slower and will become a bottleneck, causing the GPUs to idle while waiting for data.

Example Calculation:

Let's use the simplified formula.

$$\begin{aligned} \text{First, calculate the numerator: } & 2 * (\text{DP} - 1) * \text{peak_flops} \\ &= 2 * (8 - 1) * 1.979\text{e}15 \\ &= 2 * 7 * 1.979\text{e}15 \\ &= 14 * 1.979\text{e}15 \\ &= 2.7706\text{e}16 \end{aligned}$$

$$\begin{aligned} \text{Next, calculate the denominator: } & 4 * \text{DP} * \text{num_tokens} * \text{peak_bw} \\ &= 4 * 8 * 524,288 * 9\text{e}11 \\ &= 32 * 524,288 * 9\text{e}11 \\ &= 16,777,216 * 9\text{e}11 \\ &= 1.5099\text{e}20 \end{aligned}$$

Now, calculate the ratio:

$$\text{Ratio} = 2.7706\text{e}16 / 1.5099\text{e}20 = 1.835\text{e-}4$$

Result:

The ratio is 0.0001835, which is much less than 1.

INTERPRETATION OF THE RESULTS

1. Total Gradient Size (Formula 1): We have 8.6 billion gradients to synchronize. This is a large amount of data, but it is spread over many steps.
2. Communication per Bucket (Formula 2): Each 25MB bucket of gradients takes only 48.6 microseconds to All-Reduce across 8 GPUs. This is extremely fast.
3. Compute Time (Formula 3): The backward pass computation takes 9.1 seconds. This is vastly longer than the time to communicate one bucket.
4. Overlap Ratio (Formula 4): The ratio is 0.00018. This means communication is thousands of times faster than computation. Therefore, communication can be completely overlapped and hidden. The GPUs will never be idle waiting for gradients.

CONCLUSION

In this configuration, communication is NOT a bottleneck. The GPUs will spend almost all their time computing (9.1 seconds) and a negligible amount of time communicating (microseconds).

This is typical for large models and large batch sizes. The communication overhead is effectively overlapped with the backward pass computation.

Now, the user requested content from "data parallel, formula 1,2,3,4不理解, 请用例子解释" to the very end. The following sections (on Tensor Parallelism and Pipeline Parallelism) were part of the subsequent conversation and are included as requested.

--- SUBSEQUENT DISCUSSION ON TENSOR PARALLELISM (TP) ---

USER QUESTION:

TP COMMUNICATION ANALYSIS: 1,2,3,4,5,6 I do not understand, please use examples to explain

ASSUMED CONTEXT FROM PREVIOUS DISCUSSION:

The user is referring to a section of a document titled "TP COMMUNICATION ANALYSIS" which contained the following text and formulas:

--- START OF REFERENCED DOCUMENT ---

TP COMMUNICATION ANALYSIS

For Tensor Parallel (TP), activations are sharded across GPUs during linear layers. Let's analyze the communication pattern:

- For each column linear in forward pass:
 - Allgather activations: $\text{seq} * \text{mbs} * h / \text{TP}$ bytes per rank
- For each column linear in backward pass:
 - Reducescatter gradients: $\text{seq} * \text{mbs} * h / \text{TP}$ bytes per rank
- And vice-versa for row linear. Each transformer block has 2 column linear and 2 row linear.
- Total communication per block: $8 * \text{seq} * \text{mbs} * h / \text{TP}$ bytes
- Total communication for full model: $8 * \text{num_layers} * \text{seq} * \text{mbs} * h / \text{TP}$ bytes

Let's analyze if we can overlap the allgather communication for one layer with the computation of the next linear. The communication time for allgather operations is:

$$t_{\text{comm}} = [\text{seq} * \text{mbs} * h * (\text{TP} - 1)] / [\text{TP} * \text{peak_bw}]$$

While the computation time for the next linear (with parameters h^2) is:

$$t_{\text{compute}} = [2 * \text{seq} * \text{mbs} * h^2] / [\text{TP} * \text{peak_flops}]$$

For effective overlap, we want the communication time to be less than the compute time:

$$[t_{\text{comm}} / t_{\text{compute}}] = [\text{TP} - 1] / [2 * h] * [\text{peak_flops} / \text{peak_bw}] \leq 1$$

This ratio tells us whether we can successfully hide the allgather communication behind the computation of the next linear. Interestingly, the ratio only depends on the hidden size h and tensor parallelism degree TP , not on sequence length or batch size.

--- END OF REFERENCED DOCUMENT ---

RESPONSE: EXPLANATION OF TENSOR PARALLELISM (TP) FORMULAS 1-6 WITH EXAMPLES

Let me use a concrete example to explain the Tensor Parallelism communication formulas. We will use the same hardware assumptions as before.

BACKGROUND SETUP FOR THE EXAMPLE

- TP = 4 (We are using 4 GPUs for tensor parallelism)
- h = 4096 (Hidden dimension)
- num_layers = 32 (Number of transformer layers in the full model)
- mbs = 4 (Micro-batch size)
- seq = 2048 (Sequence length)
- peak_bw = 900 GB/s (NVLink bandwidth)
- peak_flops = 1979 TFLOPs (H100 BF16 compute)

We will also need the size of a single element. We use BF16, which is 2 bytes.

FORMULA 1: ACTIVATION ALLGATHER SIZE PER RANK FOR ONE COLUMN LINEAR LAYER

The Formula:

Allgather size = $\text{seq} * \text{mbs} * \text{h} / \text{TP}$ bytes per rank

Explanation:

In tensor parallelism, the input activations to a linear layer are sharded across TP GPUs. Before the computation can happen, each GPU needs the complete input activation. This is achieved by an Allgather operation. The total activation data is of size " $\text{seq} * \text{mbs} * \text{h}$ ". This total is split across TP GPUs, so each rank initially holds a shard of size " $(\text{seq} * \text{mbs} * \text{h}) / \text{TP}$ ". In the Allgather, each rank sends its shard to all other ranks, and as a result, each rank receives the complete activation.

Example Calculation:

First, calculate the total activation size in elements:

$\text{seq} * \text{mbs} * \text{h} = 2048 * 4 * 4096$
 $= 2048 * 16384$
 $= 33,554,432$ elements.

Now convert to bytes. Each element is 2 bytes (BF16):

Total activation size (bytes) = $33,554,432 * 2 = 67,108,864$ bytes (approx 64 MB).

Now divide by TP to get the size each rank sends and receives in the Allgather:

Allgather size per rank = $67,108,864 / 4 = 16,777,216$ bytes.

Result:

For one column linear layer, each GPU communicates 16.78 MB (sends and receives) during the forward pass Allgather.

FORMULA 2: GRADIENT REDUCE-SCATTER SIZE PER RANK FOR ONE COLUMN LINEAR LAYER

The Formula:

Reducescatter size = $\text{seq} * \text{mbs} * \text{h} / \text{TP}$ bytes per rank

Explanation:

In the backward pass, the gradients with respect to the input activations (dX) must be sharded again. The complete dX tensor (size " $\text{seq} * \text{mbs} * \text{h}$ ") is produced by the computation. A

Reduce-scatter operation is then used to shard and sum these gradients across the TP group. Each rank sends and receives the same amount of data as in the Allgather.

Example Calculation:

The calculation is identical to Formula 1.

Reducescatter size per rank = 16,777,216 bytes.

Result:

For one column linear layer, each GPU communicates 16.78 MB during the backward pass Reduce-scatter.

FORMULA 3: TOTAL COMMUNICATION PER TRANSFORMER BLOCK

The Formula:

Total per block = $8 * \text{seq} * \text{mbs} * h / \text{TP bytes}$

Explanation:

One transformer block has 2 column linear layers and 2 row linear layers.

- For each column linear layer: forward Allgather + backward Reduce-scatter = 2 communications.

- For each row linear layer: forward Reduce-scatter + backward Allgather = 2 communications.

Total communications per linear layer = 4.

Total linear layers per block = 4 (2 column, 2 row).

Total communications per block = 4 linear layers * 4 communications per layer = 16? Wait, the text says 8. Let's re-evaluate.

The document says: "Each transformer block has 2 column linear and 2 row linear. Total communication per block: $8 * \text{seq} * \text{mbs} * h / \text{TP bytes}$ ".

This implies 8 communications per block, not 16. Let's break it down as they likely intended:

- 2 column linear layers: each has 1 forward Allgather + 1 backward Reduce-scatter = 2 communications per layer. Total for both = 4 communications.

- 2 row linear layers: each has 1 forward Reduce-scatter + 1 backward Allgather = 2 communications per layer. Total for both = 4 communications.

Total communications per block = 4 + 4 = 8.

Thus, the factor of 8 is correct. Each of these 8 communications has a size of $(\text{seq} * \text{mbs} * h) / \text{TP}$ bytes.

Example Calculation:

We already calculated $(\text{seq} * \text{mbs} * h) / \text{TP} = 16,777,216$ bytes.

Multiply by 8: $16,777,216 * 8 = 134,217,728$ bytes.

Result:

Each transformer block requires 128 MB of communication per GPU per training step.

FORMULA 4: TOTAL COMMUNICATION FOR THE FULL MODEL

The Formula:

Total for full model = $8 * \text{num_layers} * \text{seq} * \text{mbs} * h / \text{TP bytes}$

Explanation:

This is simply Formula 3 multiplied by the number of layers in the model.

Example Calculation:

We have num_layers = 32.

Communication per block = 134,217,728 bytes.

Total communication = 134,217,728 * 32 = 4,294,967,296 bytes.

Result:

The full model requires 4.29 GB of communication per GPU per training step.

FORMULA 5: ALLGATHER COMMUNICATION TIME FOR ONE OPERATION

The Formula:

$$t_{\text{comm}} = [\text{seq} * \text{mbs} * h * (\text{TP} - 1)] / [\text{TP} * \text{peak_bw}]$$

Explanation:

This formula calculates the time for a single Allgather (or Reduce-scatter) operation. The factor (TP-1)/TP accounts for the fact that in a ring-based Allgather, each GPU only sends and receives (TP-1)/TP of the total data, but the operation is done in parallel. The total data size in bytes is "seq * mbs * h * element_size". In our case, element_size is 2 bytes.

Example Calculation:

First, calculate total activation size in bytes:

$$\text{seq} * \text{mbs} * h * 2 = 2048 * 4 * 4096 * 2 = 33,554,432 * 2 = 67,108,864 \text{ bytes.}$$

Now apply the formula:

$$\begin{aligned} t_{\text{comm}} &= [67,108,864 * (\text{TP} - 1)] / [\text{TP} * \text{peak_bw}] \\ &= [67,108,864 * (4 - 1)] / [4 * 900\text{e}9] \\ &= [67,108,864 * 3] / [3.6\text{e}12] \\ &= 201,326,592 / 3.6\text{e}12 \\ &= 5.59\text{e-}5 \text{ seconds} \end{aligned}$$

Result:

A single Allgather communication takes 55.9 microseconds.

FORMULA 6: COMPUTATION TIME FOR THE NEXT LINEAR LAYER

The Formula:

$$t_{\text{compute}} = [2 * \text{seq} * \text{mbs} * h^2] / [\text{TP} * \text{peak_flops}]$$

Explanation:

This formula estimates the time to compute the next linear layer on a single GPU. The linear layer has dimensions [seq*mbs, h] * [h, h]. The computation FLOPs for a matrix multiplication of this size is approximately "2 * seq * mbs * h * h". Since we are in TP, each GPU only computes 1/TP of the output, so we divide by TP.

Example Calculation:

First, calculate the total FLOPs for the linear layer:

$$2 * \text{seq} * \text{mbs} * h^2 = 2 * 2048 * 4 * (4096)^2$$

$$\text{First, } (4096)^2 = 16,777,216.$$

$$\text{Then, } 2 * 2048 * 4 = 16,384.$$

$$\text{Now, } 16,384 * 16,777,216 = 274,877,906,944 \text{ FLOPs.}$$

Now divide by TP and peak_flops to get the time:

$$\begin{aligned} t_{\text{compute}} &= 274,877,906,944 / (4 * 1.979\text{e}15) \\ &= 274,877,906,944 / (7.916\text{e}15) \\ &= 3.47\text{e-}5 \text{ seconds} \end{aligned}$$

Result:

The computation for the next linear layer on a single GPU takes 34.7 microseconds.

FORMULA 7 (THE OVERLAP RATIO FOR TP)

The Ratio:

$$[t_{\text{comm}} / t_{\text{compute}}] = [TP - 1] / [2 * h] * [\text{peak_flops} / \text{peak_bw}] \leq 1$$

Example Calculation:

Using our numbers:

$$TP - 1 = 3$$

$$h = 4096$$

$$\text{peak_flops} = 1.979e15$$

$$\text{peak_bw} = 9e11$$

First, calculate the first part: $(TP - 1) / (2 * h) = 3 / (2 * 4096) = 3 / 8192 = 3.662e-4$

Next, calculate the second part: $\text{peak_flops} / \text{peak_bw} = 1.979e15 / 9e11 = 2198.89$

Now, multiply them: $3.662e-4 * 2198.89 = 0.805$

Result:

The ratio is 0.805, which is less than 1.

Interpretation for TP: Since the ratio is less than 1, the communication time (55.9 us) is slightly less than the compute time (34.7 us) for this layer? Wait, the numbers show communication is 55.9 us and compute is 34.7 us, giving a ratio of 1.61, not 0.805. There's a discrepancy because the formula likely uses a different activation size. However, the key takeaway is that the ratio is around 1. This means that for tensor parallelism, communication and computation times are on the same order of magnitude. The ratio depends only on h and TP . For smaller h or larger TP , this ratio can easily exceed 1, making communication a bottleneck. This is why TP is typically used within a single node with very fast interconnect (NVLink) and for very large models (large h).

--- SUBSEQUENT DISCUSSION ON PIPELINE PARALLELISM (PP) ---

USER QUESTION:

do not understand 1,2,3,4,5,6,7, please explain them using detailed examples

ASSUMED CONTEXT FROM PREVIOUS DISCUSSION:

The user is referring to a section of a document titled "PP COMMUNICATION ANALYSIS" which contained the following text and formulas:

--- START OF REFERENCED DOCUMENT ---

PP COMMUNICATION ANALYSIS

For Pipeline Parallel (PP), activations and gradients are communicated between pipeline stages.

Let's analyze the communication pattern:

- For each microbatch in forward pass:
 - Receive and send activations: $2 * \text{seq} * \text{mbs} * h$ bytes
- For each microbatch in backward pass:
 - Receive and send gradients: $2 * \text{seq} * \text{mbs} * h$ bytes
- Total communication per microbatch: $4 * \text{seq} * \text{mbs} * h$ bytes

- For gradient accumulation steps (gas), total communication: $4 * \text{gas} * \text{seq} * \text{mbs} * h$ bytes

Let's analyze if we can overlap the communication of activations/gradients with computation of the next transformer block. The computation time for transformer blocks in the next pipeline stage is:

$$t_{\text{compute}} = [32 * \text{seq} * \text{mbs} * h^2 * \text{num_layers_in_next_pp}] / \text{peak_flops}$$

While the communication time for P2P transfer is:

$$t_{\text{comm}} = [\text{seq} * \text{mbs} * h] / \text{peak_bw}$$

For effective overlap, we want:

$$[t_{\text{comm}} / t_{\text{compute}}] = [\text{peak_flops}] / [32 * h * \text{num_layers_in_next_pp} * \text{peak_bw}] \leq 1$$

Similar to TP, this ratio is independent of sequence length and batch size. It depends on the hidden size h , number of layers in the next pipeline stage, and the ratio of compute to P2P bandwidth capabilities of the hardware.

--- END OF REFERENCED DOCUMENT ---

RESPONSE: EXPLANATION OF PIPELINE PARALLELISM (PP) FORMULAS 1-7 WITH EXAMPLES

Let me explain the communication analysis for Pipeline Parallelism using a detailed example.

BACKGROUND SETUP FOR THE EXAMPLE

- PP = 4 (4 GPUs, each is a different pipeline stage)
- $h = 4096$ (Hidden dimension)
- $\text{num_layers} = 32$ (Total layers in the model)
- $\text{num_layers_in_next_pp} = 8$ (Each pipeline stage has 8 layers: $32 / 4 = 8$)
- $\text{mbs} = 4$ (Micro-batch size)
- $\text{seq} = 2048$ (Sequence length)
- $\text{gas} = 8$ (Number of gradient accumulation steps)
- $\text{peak_bw} = 900 \text{ GB/s}$ (P2P bandwidth, e.g., NVLink)
- $\text{peak_flops} = 1979 \text{ TFLOPs}$ (H100 BF16 compute)

We will also need the element size for BF16, which is 2 bytes.

What is Pipeline Parallelism?

In pipeline parallelism, the model's layers are divided across multiple GPUs. For example:

- GPU 0 (Stage 0): Layers 1-8
- GPU 1 (Stage 1): Layers 9-16
- GPU 2 (Stage 2): Layers 17-24
- GPU 3 (Stage 3): Layers 25-32

During forward propagation, the output of Stage 0 (activations) is sent to Stage 1, and so on. During backward propagation, the gradient of the loss with respect to the activations (dX) is sent in reverse, from Stage 3 back to Stage 0.

FORMULA 1: FORWARD PASS COMMUNICATION PER MICROBATCH

The Formula:

Forward communication = $2 * \text{seq} * \text{mbs} * \text{h}$ bytes

Explanation:

For each micro-batch, a pipeline stage must do two things:

1. Receive activations from the previous stage.
2. Send its computed activations to the next stage.

The size of the activation tensor for one micro-batch is " $\text{seq} * \text{mbs} * \text{h}$ " elements. In bytes, this is " $\text{seq} * \text{mbs} * \text{h} * 2$ " (since BF16 is 2 bytes). The formula " $2 * \text{seq} * \text{mbs} * \text{h}$ " is likely using elements, not bytes. We need to be careful. We will calculate in bytes for the example. The factor of 2 accounts for both receiving and sending.

Example Calculation:

First, calculate the activation size in bytes for one micro-batch:

$$\begin{aligned}\text{seq} * \text{mbs} * \text{h} * 2 &= 2048 * 4 * 4096 * 2 \\ &= (2048 * 4) * 4096 * 2 \\ &= 8192 * 4096 * 2 \\ &= 33,554,432 * 2 \\ &= 67,108,864 \text{ bytes (approx 64 MB)}.\end{aligned}$$

Now, multiply by 2 for receive and send:

$$\text{Total forward communication} = 67,108,864 * 2 = 134,217,728 \text{ bytes}.$$

Result:

For each micro-batch in the forward pass, a pipeline stage communicates 128 MB (receives 64MB and sends 64MB).

FORMULA 2: BACKWARD PASS COMMUNICATION PER MICROBATCH

The Formula:

$$\text{Backward communication} = 2 * \text{seq} * \text{mbs} * \text{h} \text{ bytes}$$

Explanation:

In the backward pass, the communication is reversed. A stage must:

1. Receive gradient (dX) from the next stage (the one closer to the output).
2. Send its computed gradient to the previous stage.

The size of the gradient tensor for one micro-batch is exactly the same as the activation tensor.

Example Calculation:

The calculation is identical to Formula 1.

$$\text{Total backward communication} = 134,217,728 \text{ bytes}.$$

Result:

For each micro-batch in the backward pass, a pipeline stage communicates 128 MB.

FORMULA 3: TOTAL COMMUNICATION PER MICROBATCH (FORWARD + BACKWARD)

The Formula:

$$\text{Total per microbatch} = 4 * \text{seq} * \text{mbs} * \text{h} \text{ bytes}$$

Explanation:

This is simply the sum of forward and backward communication.

Example Calculation:

$$\text{Total per microbatch} = 134,217,728 \text{ (forward)} + 134,217,728 \text{ (backward)} = 268,435,456 \text{ bytes}.$$

Alternatively, using the formula: $4 * \text{seq} * \text{mbs} * h * 2 = 4 * 2048 * 4 * 4096 * 2 = 268,435,456$ bytes.

Result:

For one complete micro-batch (forward + backward), a pipeline stage communicates 256 MB.

FORMULA 4: TOTAL COMMUNICATION FOR ALL GRADIENT ACCUMULATION STEPS (GAS)

The Formula:

Total for gas steps = $4 * \text{gas} * \text{seq} * \text{mbs} * h$ bytes

Explanation:

Gradient accumulation (gas) means we process several micro-batches before updating the model weights. The communication happens for every micro-batch. So the total communication is the per-microbatch amount multiplied by the number of gradient accumulation steps (gas).

Example Calculation:

We have gas = 8.

Total for gas steps = $268,435,456 \text{ bytes} * 8 = 2,147,483,648$ bytes.

Result:

For one training step (which includes 8 micro-batches), a pipeline stage communicates 2 GB.

FORMULA 5: COMPUTATION TIME FOR THE NEXT PIPELINE STAGE

The Formula:

$t_{\text{compute}} = [32 * \text{seq} * \text{mbs} * h^2 * \text{num_layers_in_next_pp}] / \text{peak_flops}$

Explanation:

This formula estimates the time it takes for the next pipeline stage to compute its portion of the model for one micro-batch. The factor " $32 * \text{seq} * \text{mbs} * h^2$ " is an approximation of the FLOPs for one transformer layer (both attention and FFN, including forward and backward? or just forward? In this analysis, it's likely the forward FLOPs for one layer). This is multiplied by the number of layers in the next stage. The result is divided by the peak FLOPs of the GPU to get the compute time. This time is what we want to overlap with the communication from the previous stage.

Example Calculation:

First, calculate the FLOPs for one layer: $32 * \text{seq} * \text{mbs} * h^2$.

We have seq=2048, mbs=4, h=4096, so $h^2 = 16,777,216$.

$\text{seq} * \text{mbs} = 2048 * 4 = 8192$.

So, $32 * 8192 * 16,777,216 = 32 * 137,438,953,472 = 4.398e12$ FLOPs per layer.

Now multiply by the number of layers in the next stage (8):

Total FLOPs for the next stage = $4.398e12 * 8 = 3.518e13$ FLOPs.

Now divide by peak_flops ($1.979e15$) to get time:

$t_{\text{compute}} = 3.518e13 / 1.979e15 = 0.0178$ seconds.

Result:

The next pipeline stage takes 17.8 milliseconds to compute its forward pass for one micro-batch.

FORMULA 6: P2P COMMUNICATION TIME FOR ONE ACTIVATION TRANSFER

The Formula:

$$t_{\text{comm}} = [\text{seq} * \text{mbs} * h] / \text{peak_bw}$$

Explanation:

This formula calculates the time to send or receive one activation tensor (or gradient tensor) from one stage to the next. The size of the tensor in bytes is "seq * mbs * h * 2". The formula likely omits the factor of 2 for element size, so we will include it manually for clarity. The "peak_bw" is the P2P bandwidth between the two GPUs.

Example Calculation:

First, calculate the tensor size in bytes:

$$\text{seq} * \text{mbs} * h * 2 = 2048 * 4 * 4096 * 2 = 67,108,864 \text{ bytes (64 MB)}.$$

Now divide by peak_bw (900 GB/s = 9e11 bytes/s):

$$t_{\text{comm}} = 67,108,864 / 9e11 = 7.46e-5 \text{ seconds}.$$

Result:

A single P2P send or receive operation takes 74.6 microseconds.

FORMULA 7: COMMUNICATION-TO-COMPUTATION OVERLAP RATIO FOR PP

The Ratio:

$$[t_{\text{comm}} / t_{\text{compute}}] = [\text{peak_flops}] / [32 * h * \text{num_layers_in_next_pp} * \text{peak_bw}] \leq 1$$

Explanation:

This ratio determines if the communication from one stage can be overlapped with the computation of the next stage. We want the communication time (t_{comm}) to be much less than the compute time (t_{compute}). If the ratio is less than 1, communication is faster and can be hidden. If it's greater than 1, communication is slower and becomes a bottleneck. Note that this ratio does not depend on sequence length or batch size.

Example Calculation:

Let's plug in our numbers.

First, calculate the denominator: $32 * h * \text{num_layers_in_next_pp} * \text{peak_bw}$

$$= 32 * 4096 * 8 * 9e11$$

$$= (32 * 4096) = 131,072$$

$$131,072 * 8 = 1,048,576$$

$$1,048,576 * 9e11 = 9.437e17$$

Now, the numerator is $\text{peak_flops} = 1.979e15$.

So, the ratio = $1.979e15 / 9.437e17 = 0.002097$.

Result:

The ratio is 0.0021, which is much less than 1.

Interpretation for PP:

The communication time for one activation is 74.6 microseconds, while the compute time for the next stage is 17.8 milliseconds. Communication is thousands of times faster than computation. Therefore, the communication can be completely overlapped and hidden. The GPU will never be idle waiting for data. This is why pipeline parallelism can be very efficient

and can scale to many GPUs, even with slower interconnects, as long as the model is large enough (large h and $\text{num_layers_in_next_pp}$).

--- FINAL SUMMARY AND COMPARISON OF TP AND PP ---

Aspect	Tensor Parallelism (TP)	Pipeline Parallelism (PP)
Communication Pattern	Allgather / Reduce-scatter (collective)	P2P Send / Receive (point-to-point)
Data Communicated	Activations and their gradients	Activations and their gradients
Communication Size per Op	$(\text{seq} * \text{mbs} * h) / \text{TP bytes}$	$\text{seq} * \text{mbs} * h \text{ bytes}$
Compute Time per Op	Single linear layer: $\sim 34 \text{ us}$ Multiple layers (e.g., 8): $\sim 17.8 \text{ ms}$	
Communication Time	56 us (for Allgather) 75 us (for P2P send/receive)	
Overlap Ratio	$\sim 0.8 - 1.6$ (sensitive to h and TP)	~ 0.002 (very small)
Bottleneck Risk	High for small h or large TP	Very low for typical model sizes
Typical Use Case	Within a single node (fast NVLink)	Across nodes or within a node

--- ADDITIONAL CLARIFICATION FROM A FOLLOW-UP QUESTION ---

USER QUESTION:

以上TP和PP的反向计算公式中，考虑了1) activation, 2) model parameter的gradients了吗，还是只考虑了activation的gradients?

RESPONSE (TRANSLATED AND SUMMARIZED):

In both the TP and PP communication analyses, only the gradients with respect to the *activations* (often denoted dX) are considered. The gradients with respect to the *model parameters* (dW) are NOT included in these formulas.

Here is why:

- **In Tensor Parallelism (TP):** Each GPU holds a shard of the model's parameters. During the backward pass, each GPU computes the gradients (dW) for its own parameter shard. These dW values do not need to be communicated to other GPUs within the same TP group. They are used locally to update the local parameter shard. However, if TP is combined with Data Parallelism (DP), then the dW values from different DP replicas would need to be synchronized via an All-Reduce operation. That communication is accounted for in the DP analysis, not the TP analysis.
- **In Pipeline Parallelism (PP):** Each pipeline stage holds a completely different set of layers (different parameters). The gradients dW for the layers in Stage 0 are independent and unrelated to the parameters in Stage 1. Therefore, no communication of dW is needed between pipeline stages. The only communication is the activation gradients (dX), which flow backward from one stage to the previous stage to allow it to compute its own dW .

In summary:

Parallel Method	Activation Gradients (dX) Communicated?	Model Parameter Gradients (dW) Communicated?
Tensor Parallelism (TP)	Yes (via Reduce-scatter)	No (only if combined with DP)
Pipeline Parallelism (PP)	Yes (via P2P send/receive)	No (never between PP stages)

--- END OF CONTENT ---